유비샘 5차 세미나

https://hwanheejung.tistory.com/42

https://velog.io/@limsubin/STOMP%EC%9D%84-%EC%95%8C%EC%95%84%EB%B3%B4%EA%B3%A0-%EA%B5%AC%ED%98%84%ED%95%B4%EB%B3%B4%EC%9E%90

> 💡 이번 세미나는 해당 URL 을 참조하면 좋습니다.

Restful Query

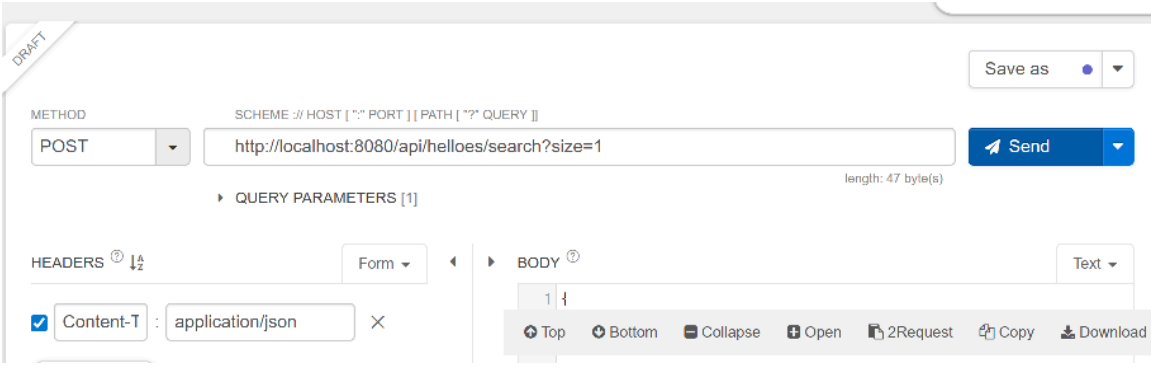
- 의존성을 주입받아 만든 Repository에서 pagenation도 지원합니다.

  예시로 talend로 테스트를 해보겠습니다.

  아래 사진과 같이

  METHOD: POST

  SCHEME: http://localhost:8080/api/helloes/search?size=1

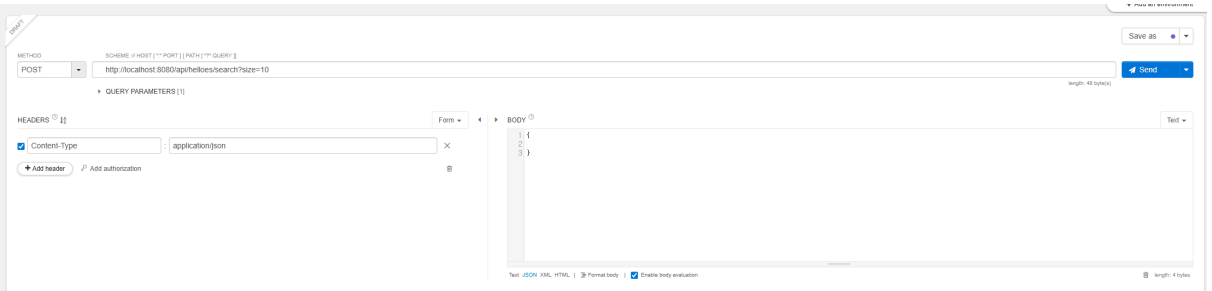


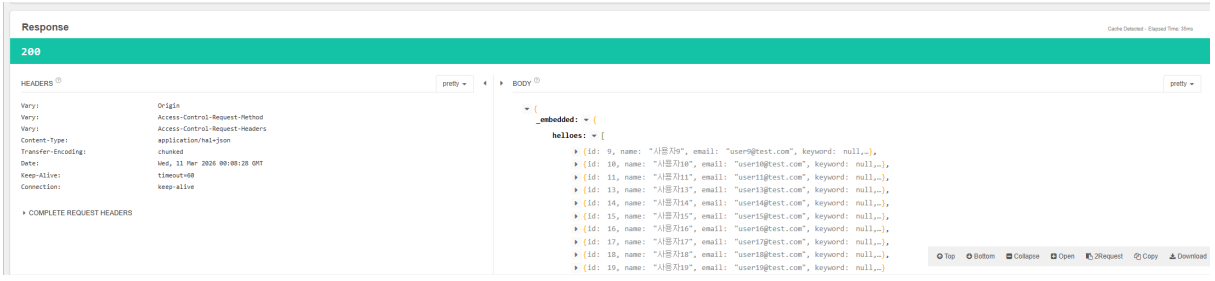
- 이런 식으로 설정 후 Send를 보내게 되면 아래 사진과 같이

  size 뒤에 지정한 숫자의 개수만큼 값을 받아오게 됩니다.

  Default 설정 값은 20입니다.

- 만약 size값을 늘리면 그 만큼 많은 값을 받아올 수 있습니다.



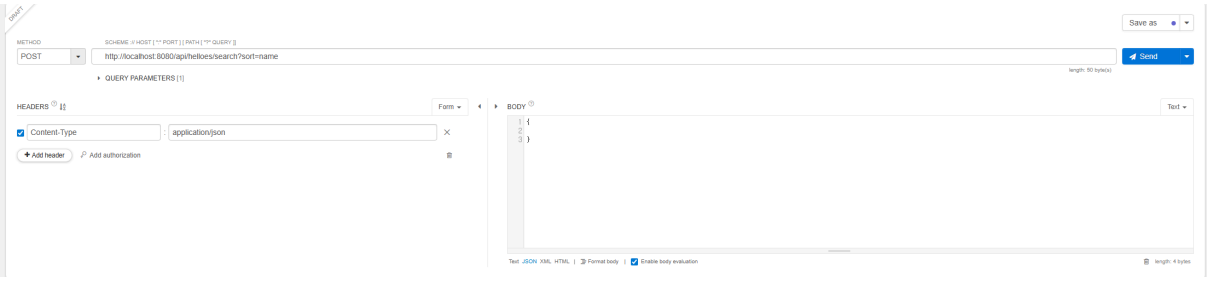


- 다음 지원 기능은 sort입니다.

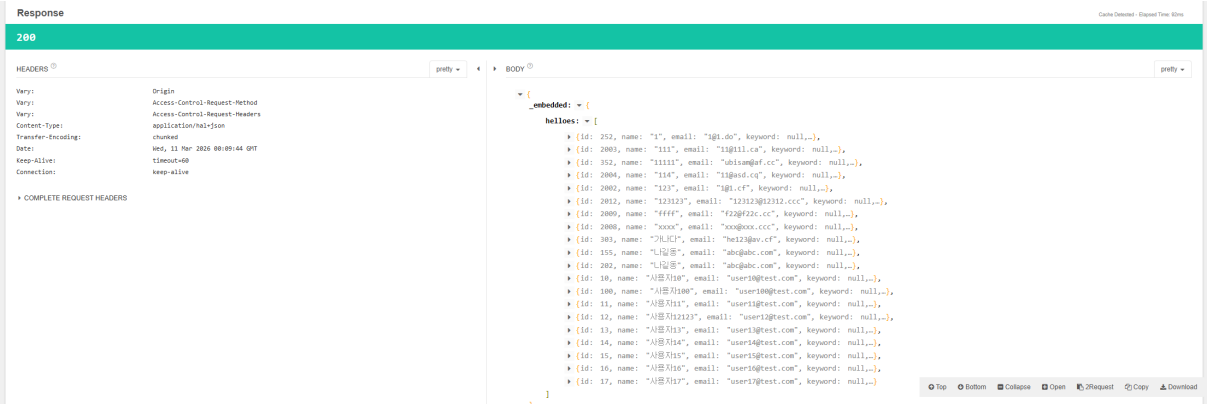
  sort 뒤에 설정한 파라미터 값으로 정렬해서 불러옵니다.

  METHOD: POST

  SCHEME: http://localhost:8080/api/helloes/search?size=1



- 아래와 같이 결과값을 정렬해서 불러온다는 것을 확인할 수 있습니다.



Stomp

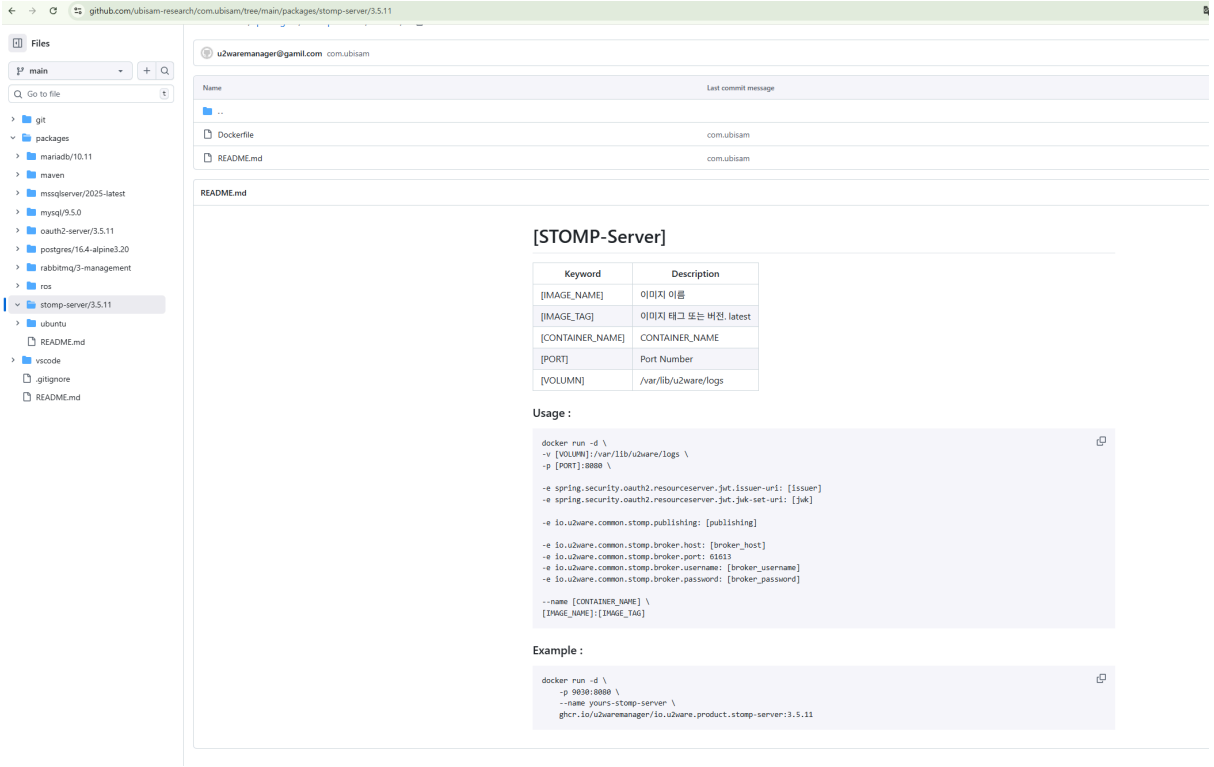
Docker - Stomp 서버

- Stomp를 이해하기 전 먼저 테스트를 해보는 과정입니다.

  빠른 학습을 위해 링크 이동은 패스합니다. 아래 링크를 클릭해주시길 권장드립니다.

  아래 사진으로 이동되면 정상입니다.

https://github.com/ubisam-research/com.ubisam/tree/main/packages/stomp-server/3.5.11



- 여기서 맨 아래 Example에 있는 명령어를 복사합니다. 그 후 1줄로 만들어주는 작업을 합니다.

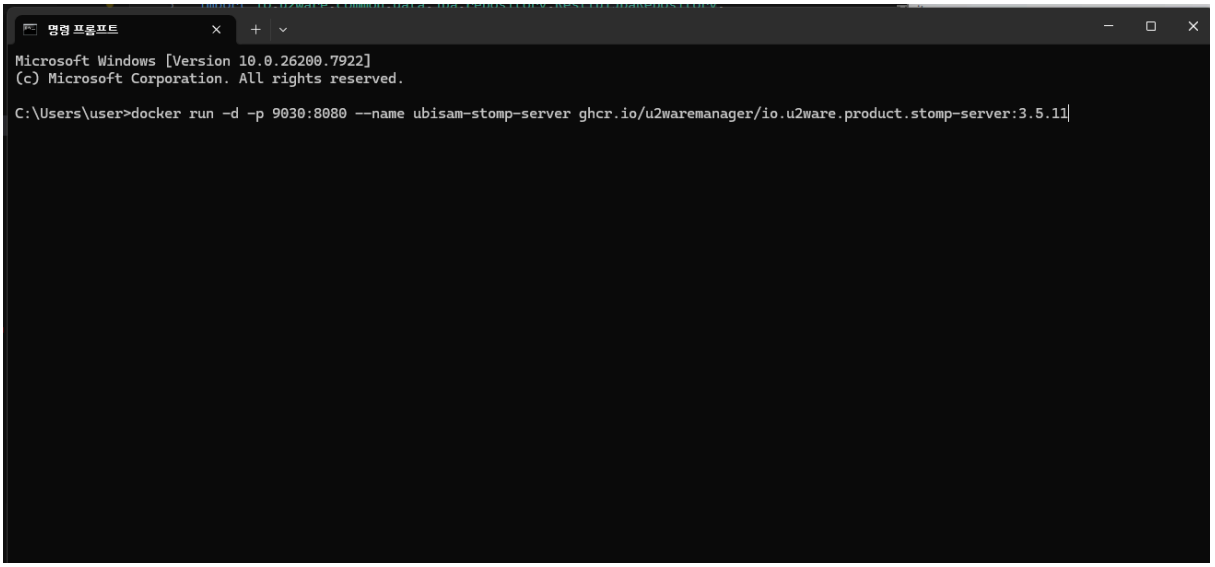
  아래 명령어는 작업이 완료 된 후 명령어입니다. 귀찮으시면 제꺼 그대로 복사하시면 됩니다.

  저는 구별이 편하게 container 이름을 ubisam-stomp-server 라고 명명하였습니다.

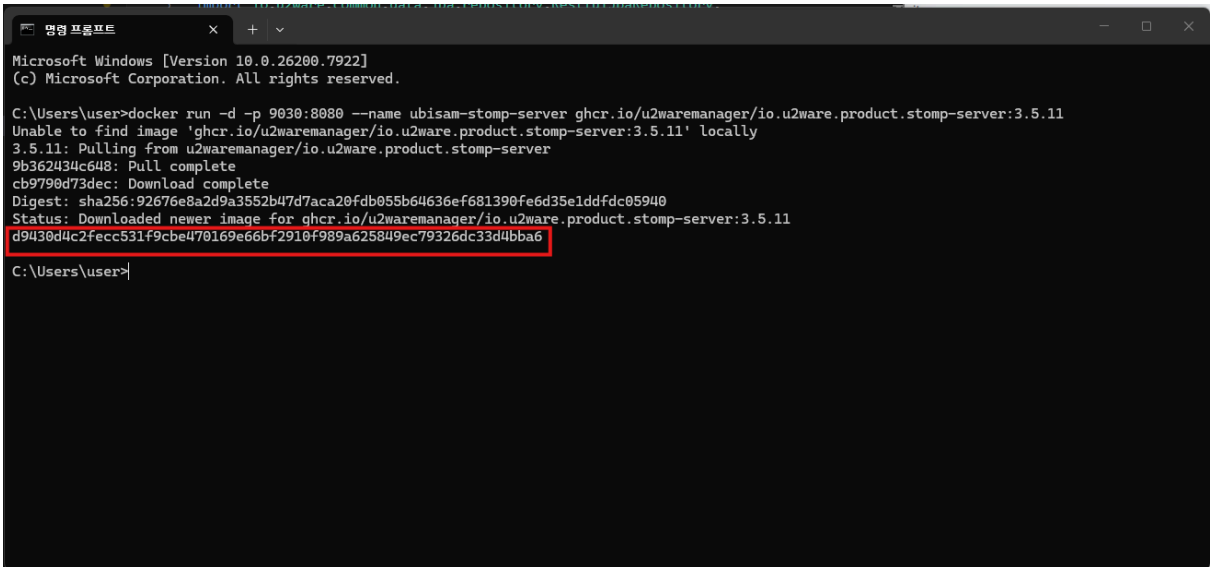
  포트(현재: 9030)와 이름(현재: ubisam-stomp-server)은 원하시는대로 바꾸셔도 무방합니다.

> 💡 docker run -d -p 9030:8080 --name ubisam-stomp-server ghcr.io/u2waremanager/io.u2ware.product.stomp-server:3.5.11

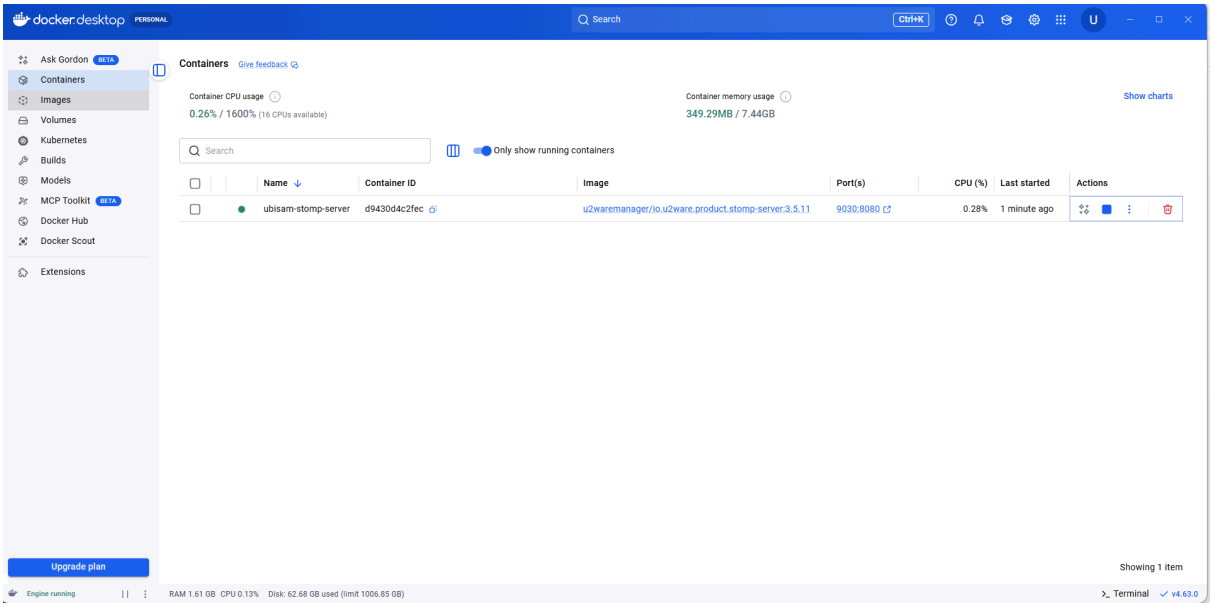
- 아래 사진과 같이 명령프롬포트 혹은 window powershell 등을 연 후 해당 명령어를 입력합니다.



- 다운과 docker 컨테이너 설정이 완료 된 후 아래 사진과 같이 container가 열리게 됩니다.

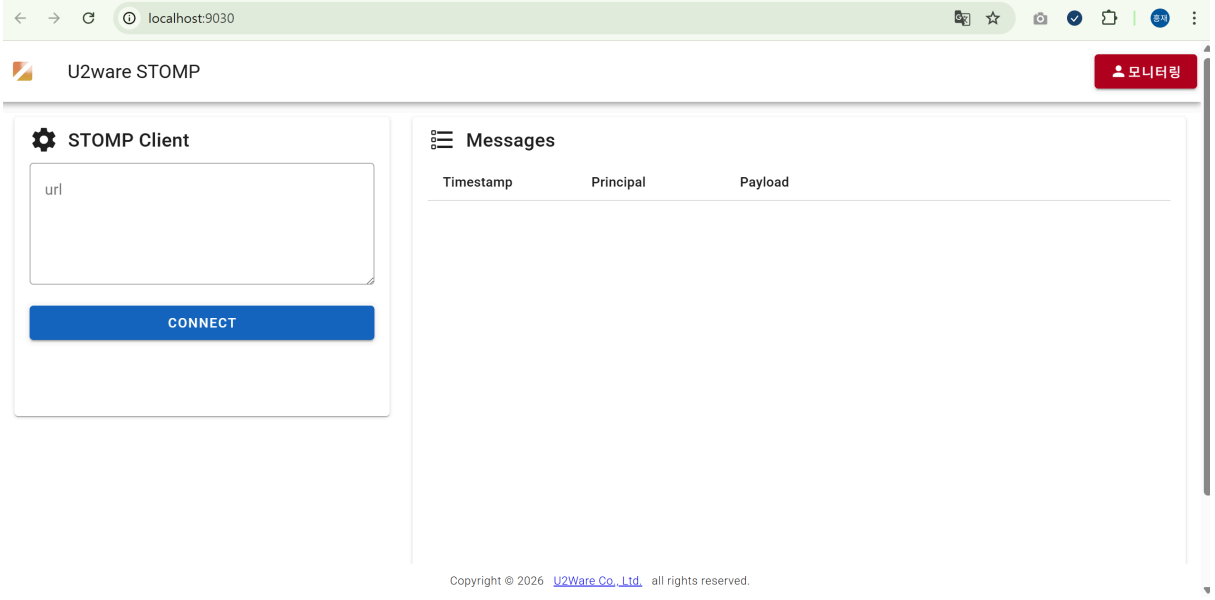


- docker.desktop으로 확인 시 아래 사진과 같이 정상적으로 작동하는 것을 확인하실 수 있습니다.



- 그 후 테스트를 해보기 위해 주소창에 localhost:9030 를 입력합니다.

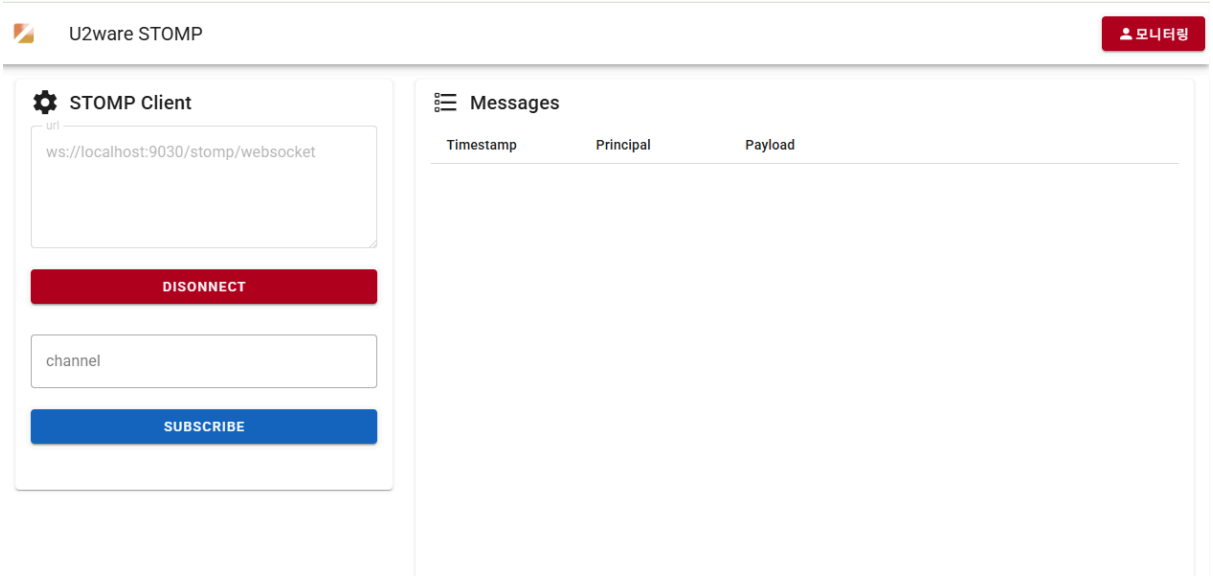
  그럼 해당 브라우저로 이동하게 됩니다.



- 처음 url 창에는 ws://localhost:9030/stomp/websocket을 입력 후 connect를 눌러 연결합니다.

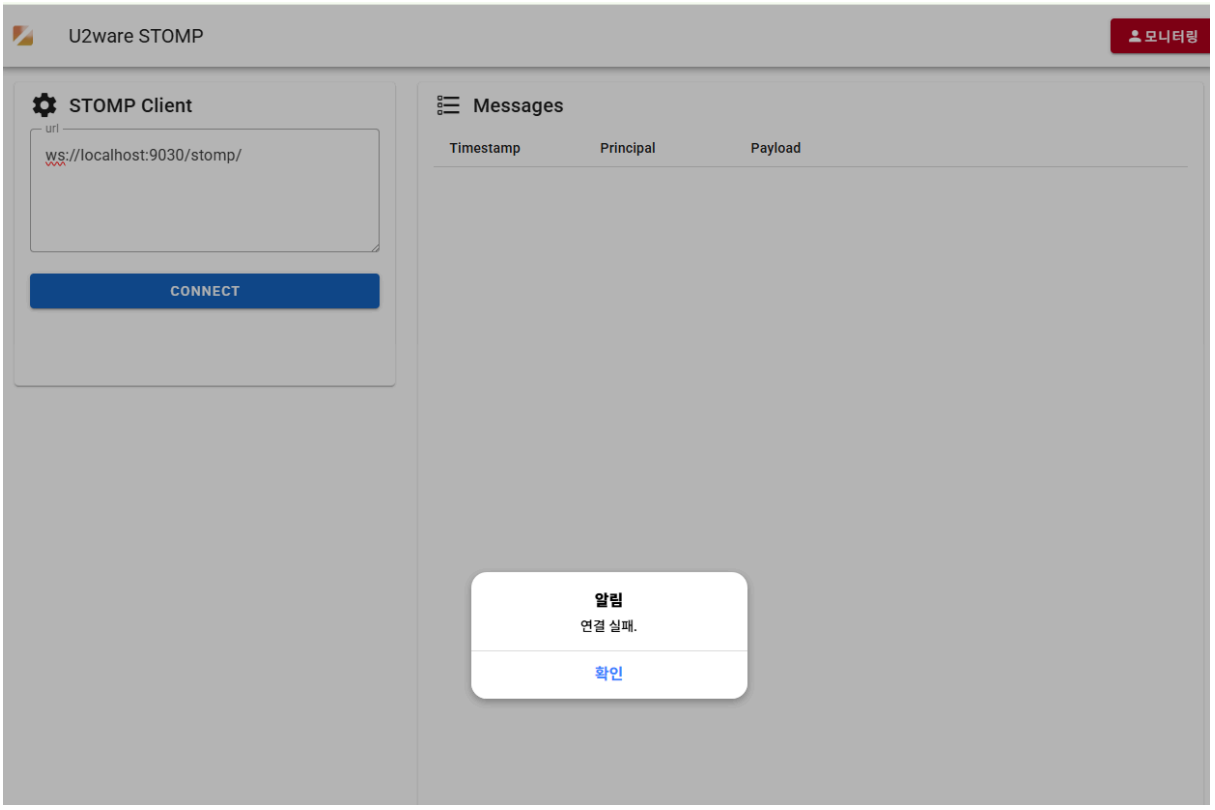
  아래 사진과 같이 연결되면 성공입니다.

> 💡 ws://localhost:9030/stomp/websocket



- 만약 아래 사진과 같이 연결 실패가 뜨면 docker가 실행이 정상적으로 이루어지지 않았거나

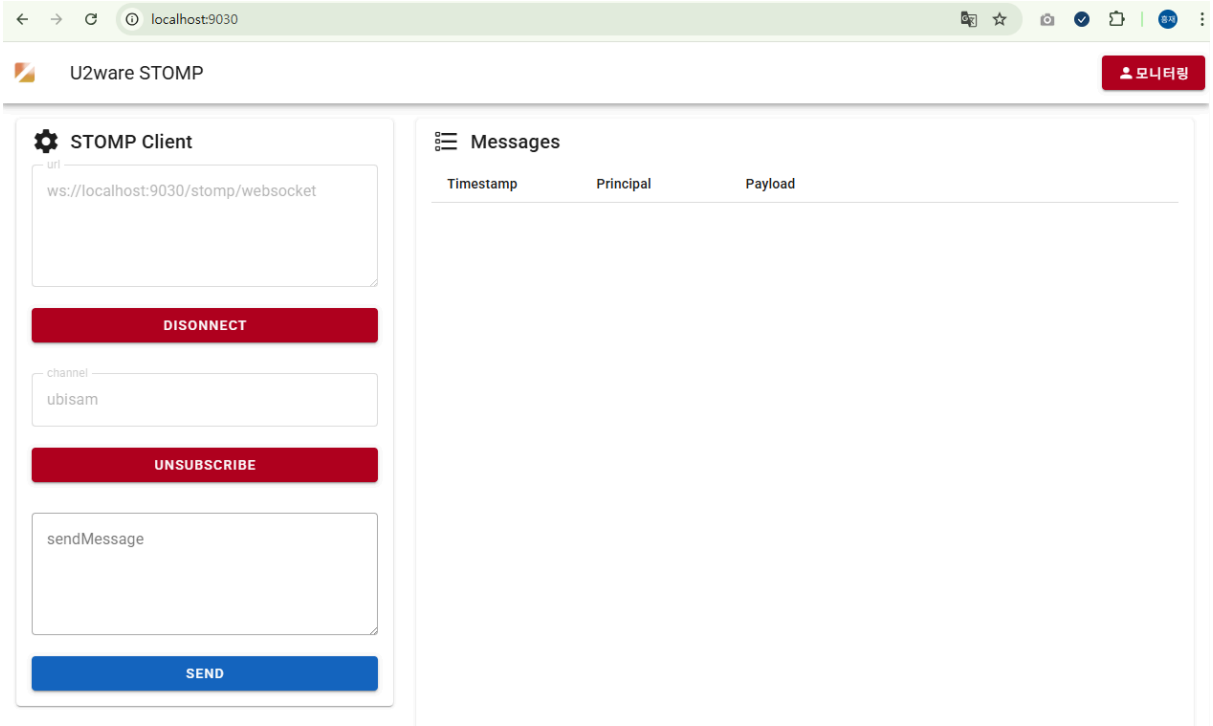
  주소 입력에 오타가 난 것입니다.



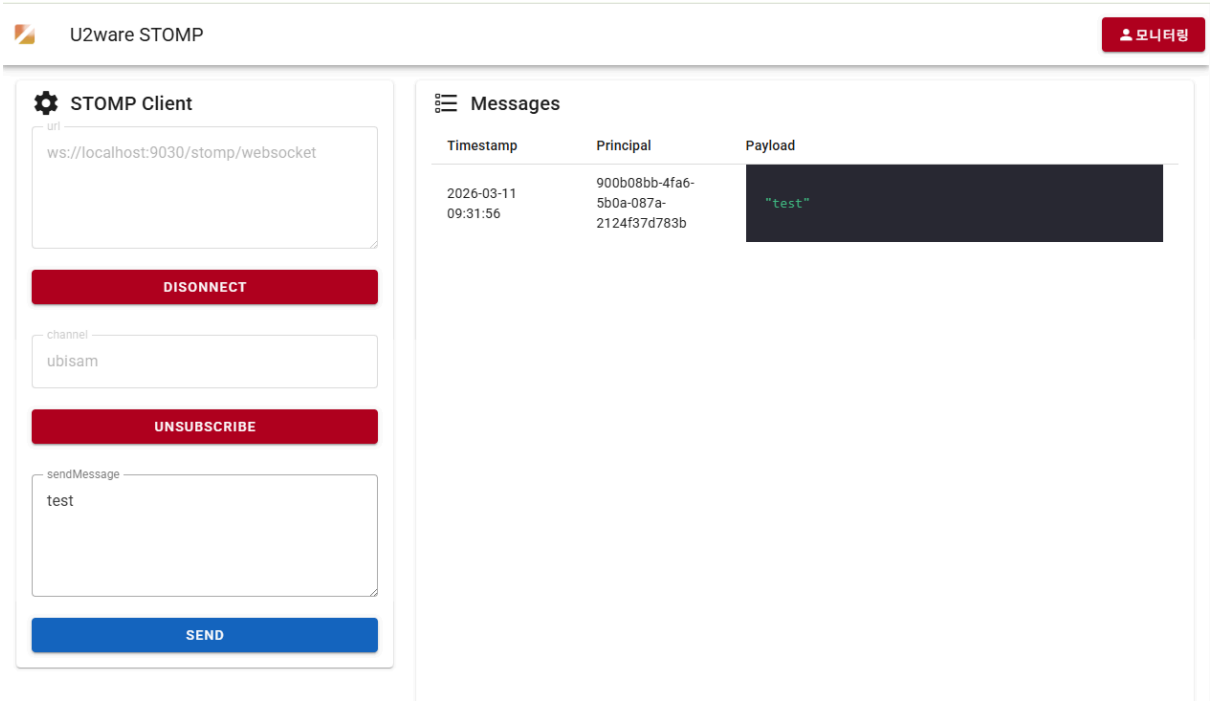
- 그 후 channel은 원하는 채널이름 아무거나 입력합니다.

  저는 세미나대로 ubisam 으로 들어가보겠습니다.

  채널 입력 후 subscribe를 누르게 되면 전 ubisam 채널을 구독한 상태가 됩니다.

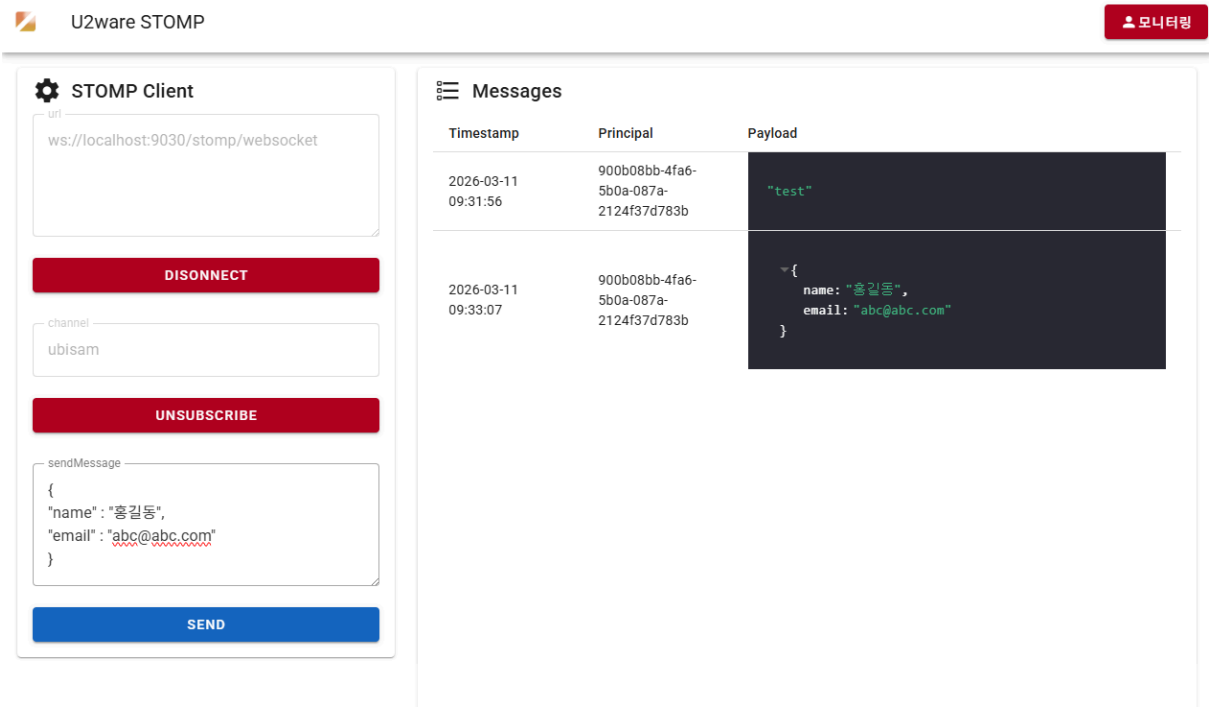


- 그 후 원하는 메시지를 send 해보겠습니다.



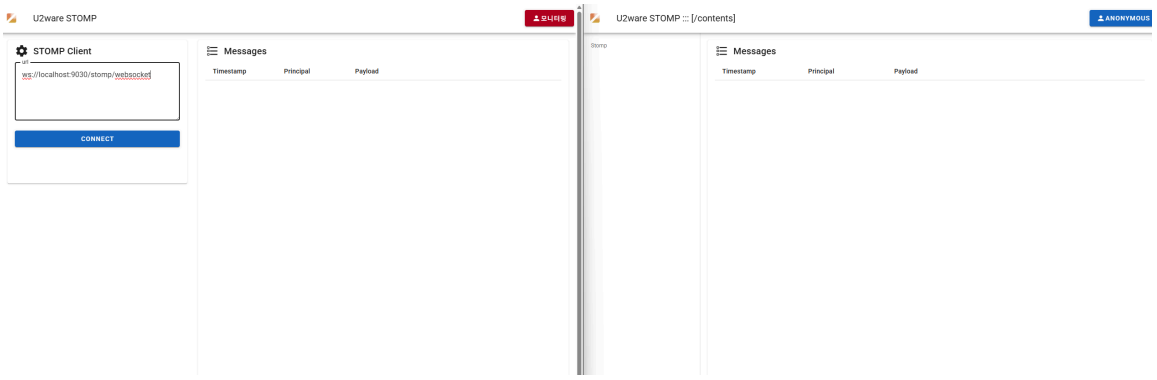
- test 입력 후 send 버튼을 누르면 위 사진과 같이 messages에 정상적으로 timestamp, principal, payload가 찍히게 됩니다.

  json 형식으로 message를 보내보겠습니다.

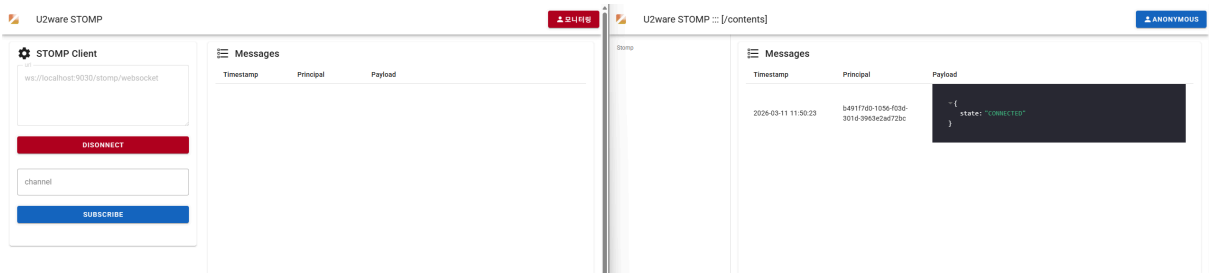


- 위 사진과 같이 payload에 json이 형태로 message가 찍히는걸 확인 가능합니다.

- 인원이 몇 명 접속했는지 확인하기 위해 2개의 브라우저로 테스트를 해봅니다.
  왼쪽 브라우저는 접속, 오른쪽 브라우저는 모니터링 상태입니다.

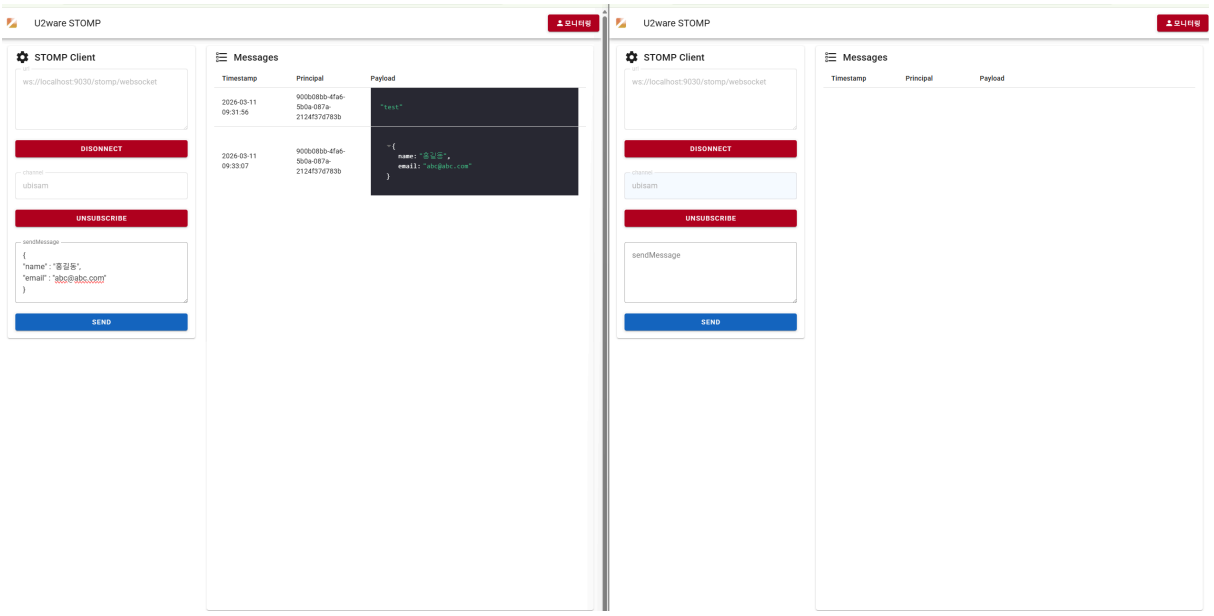


- Connect를 통해 접속을 테스트해봅니다.

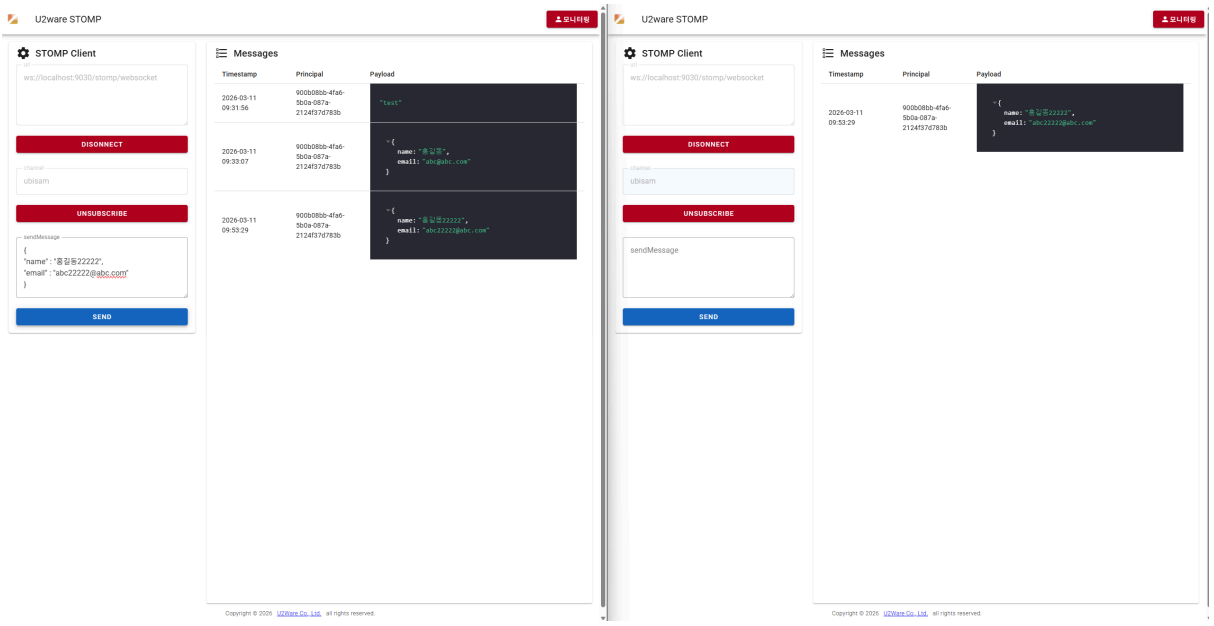


- 위 사진과 같이 정상적으로 메시지가 뜨는걸 확인해 볼 수 있습니다.

- 동시에 2개의 브라우저로 정상적으로 message가 오는지 테스트해보겠습니다.



- 위 사진과 같이 2개의 브라우저를 열어서 왼쪽 브라우저에서 다시 message를 보내보겠습니다.



- 위 사진과 같이 오른쪽 브라우저에서도 message가 뜨는걸 확인이 가능합니다.

STOMP 이론

- **STOMP (Simple Text Oriented Messaging Protocol)**: WebSocket(또는 TCP) 위에서 **메시지 전송을 목적지(destination)로 라우팅**할 수 있게 해주는 **텍스트 기반 메시징 프로토콜** 입니다.

프로토콜(통신 규약) 기본

- 프로토콜은 **서버와 클라이언트**가 있어야 성립합니다.
- 예시
  - HTTP server ↔ HTTP client(웹브라우저)
  - STOMP server(예: Docker 컨테이너) ↔ STOMP client(브라우저 등)

왜 WebSocket이 필요한가

- **HTTP는 stateless**라서 요청-응답 중심 통신에 적합합니다.
- **WebSocket은 stateful(연결 유지)**이라서 서버가 클라이언트로 **실시간 push**를 할 수 있습니다.

Pub/Sub 흐름

- 개념 흐름: **publish → (broker의 pipe/채널) → subscribe**

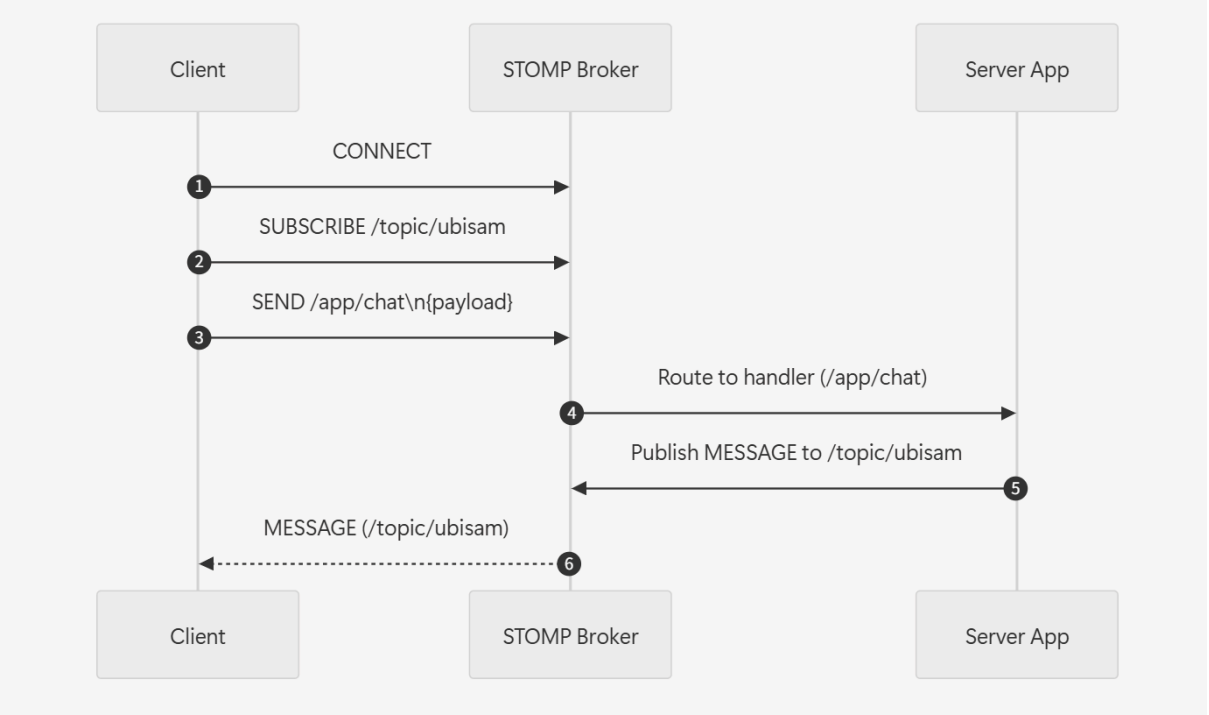
- **AMQP (Advanced Message Queuing Protocol)** 도 이런 **pub-sub** 메시징 모델을 사용합니다.

WebSocket만으로 부족한 점 → STOMP가 해결

- WebSocket은 "연결된 모든 쪽에 그냥 쏘는" 수준으로 쓰면 **목적지별 라우팅**이 애매해질 수 있습니다.
- STOMP는 WebSocket 위에 **destination(주소) 기반 라우팅 규칙**을 얹어서 해결합니다.
  - 요약: **STOMP = WebSocket + 라우팅(메시징 규약)**

STOMP의 주소(destination) 예시

- 서버는 내부에 여러 "파이프(채널)"를 두고, 각 파이프에 주소를 부여합니다.
  - `/topic/{이름}` : 보통 브로드캐스트(구독자에게 전달)
  - `/app/{이름}` : 보통 서버 애플리케이션으로 들어오는 엔드포인트(서버가 규칙/이름 결정)



Stomp Backend Client

- 먼저 pom.xml에 의존성을 주입해야 합니다.

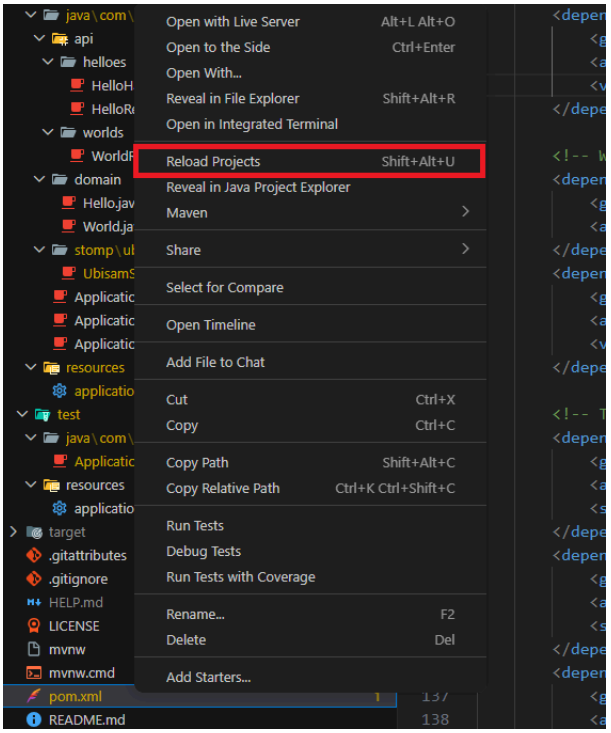
아래 의존성을 주입합니다. 기존에 u2ware-common-stomp 는 WebSocket 주석 밑으로 옮겨줍니다.

```
<!-- WebSocket -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
<dependency>
    <groupId>io.u2ware.common</groupId>
    <artifactId>u2ware-common-stomp</artifactId>
    <version>3.5.11</version>
</dependency>
```

- 아래 사진과 같이 입력하시면 됩니다.

```
<!-- io.u2ware -->
<dependency>
    <groupId>io.u2ware.common</groupId>
    <artifactId>u2ware-common-oauth2</artifactId>
    <version>3.5.11</version>
</dependency>

<!-- WebSocket -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
<dependency>
    <groupId>io.u2ware.common</groupId>
    <artifactId>u2ware-common-stomp</artifactId>
    <version>3.5.11</version>
</dependency>

<!-- Test -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

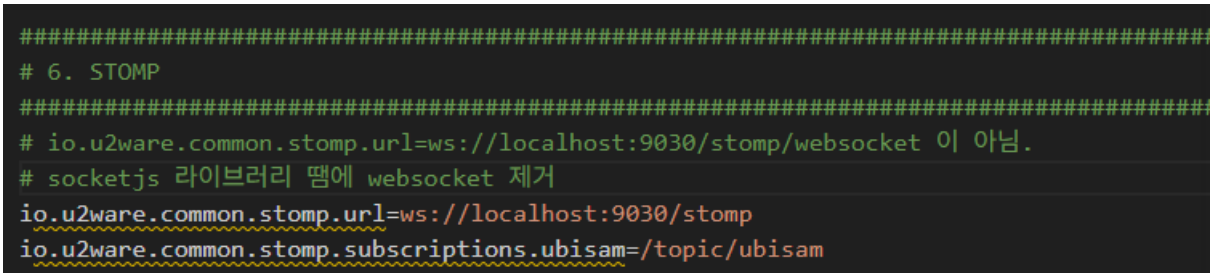
- 그 후 pom.xml을 우클릭 하여 Reload Projects 로 프로젝트를 재부팅 합니다.
  만약 Reload 완료 후에도 의존성 주입이 이루어지지 않았다면 VSCode를 재부팅 합니다.



- 다음으로 application.properties 입니다.
  아래 설정을 추가해줍니다.

```
# io.u2ware.common.stomp.url=ws://localhost:9030/stomp/websocket 이 아님.
# socketjs 라이브러리 땜에 websocket 제거
io.u2ware.common.stomp.url=ws://localhost:9030/stomp
io.u2ware.common.stomp.subscriptions.ubisam=/topic/ubisam
```

- 아래 사진과 같이 추가하시면 됩니다.



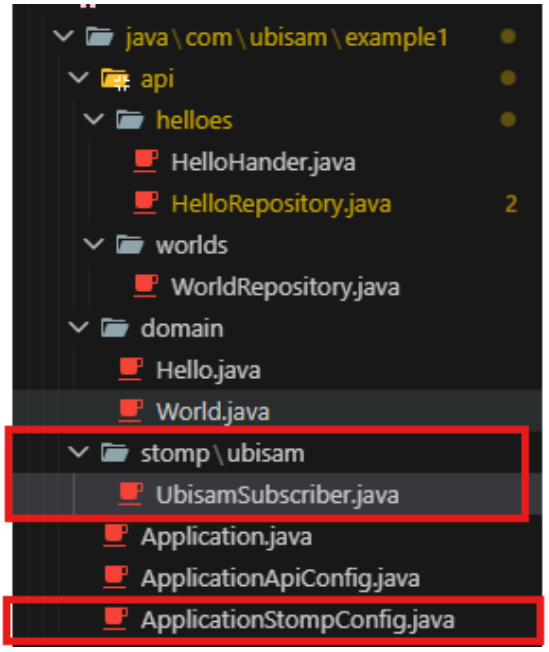
- COC 규약을 준수하여 stomp 폴더와 stomp config를 만들겠습니다.

  Application.java, ApplicationApiConfig.java 와 같은 위계에

  ApplicationStompConfig.java 파일을 생성합니다.

- 그 후 domain 과 같은 위계에 stomp폴더를 만들고 해당 폴더 안에

  ubisam이라는 폴더를 만든 후 UbisamSubscriber.java 파일을 생성합니다.



- ApplicationStompConfig.java를 연 후 아래 코드를 작성합니다.

  Config 파일이기에 @Configuration 어노테이션을 붙여줬습니다.

```
import org.springframework.context.annotation.Configuratio
n;

import io.u2ware.common.stomp.client.config.EnableWebsocket
StompClient;

@Configuration
@EnableWebsocketStompClient
public class ApplicationStompConfig {

}
```

- 아래 사진과 같이 작성하시면 됩니다.

```
example1 > src > main > java > com > ubisam > example1 > ApplicationStompConfig.java > ...
1   package com.ubisam.example1;
2
3   import org.springframework.context.annotation.Configuration;
4
5   import io.u2ware.common.stomp.client.config.EnableWebsocketStompClien
6
7   @Configuration
8   @EnableWebsocketStompClient
9   public class ApplicationStompConfig {
10
11  }
12
```

- UbisamSubscriber.java 파일을 연 후 먼저 WebSocketStompClientHandler 인터페이스를

  구현하기 위해 implements로 받아옵니다.

  그럼 아래 사진과 같이 빨간줄이 그어진 상태로 변합니다.

```
package com.ubisam.example1.stomp.ubisam;

import io.u2ware.common.stomp.client.WebsocketStompClientHandler;

public class UbisamSubscriber implements WebsocketStompClientHandler{

}
```

> 💡 인터페이스는 기능의 규약이기 때문에 이를 구현하는 클래스는
> 모든 메서드를 반드시 구현해야 합니다.

자바 인터페이스 간략 설명

- **정의**: 인터페이스는 *기능의 규약(계약, contract)* 입니다.
- **의미**: "이 인터페이스를 구현(implements)하는 클래스는 **정해진 메서드들을 반드시 제공해야 한다**"라고 강제합니다.
- **역할**
  - 구현체가 달라도 **같은 방식으로 호출**할 수 있게 합니다. (다형성)
  - 기능이 커져도 규약을 중심으로 **확장/교체**가 쉬워집니다.

    - 팀 개발에서 “무엇을 제공해야 하는지”를 명확히 합니다.
- **주의**: 인터페이스를 구현하면, 컴파일 오류가 나지 않도록 **모든 추상 메서드를 구현**해야 합니다.
- 인터페이스는 이렇게 말합니다.

> "이 인터페이스를 사용하는 클래스는 반드시 이 기능들을 제공해야 한다."

예시 (Payment 인터페이스)

- **상황**: 결제 시스템을 만든다고 가정합니다.
- **핵심**: `Payment`를 `implements` 하는 클래스는 **반드시** `pay()`**를 구현**해야 합니다.

```
// 결제 기능 규약
interface Payment {
    void pay();
}
```

정상 구현 예시

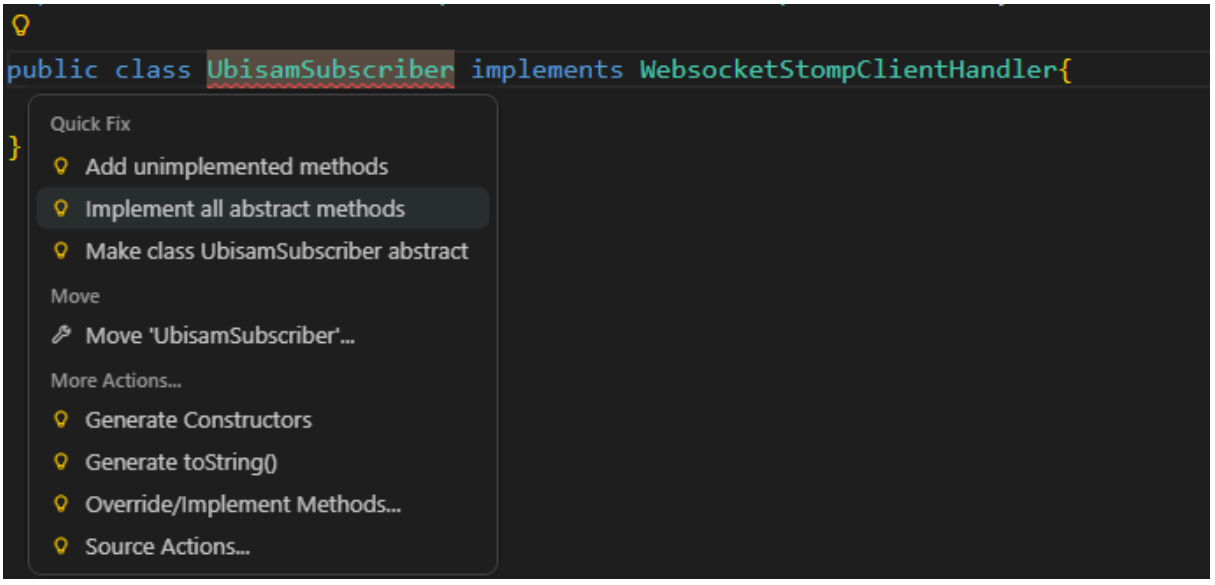
```
class CardPayment implements Payment {
    @Override
    public void pay() {
        System.out.println("카드 결제");
    }
}
```

구현 누락 시(컴파일 오류)

```
class CardPayment implements Payment {
    // pay() 미구현
}
```

> 💡 `Payment` 는 "결제 기능을 제공해야 한다"는 **규약**이므로, `pay()` 가 없으면 구현 클래스가 성립하지 않아 컴파일 에러가 발생합니다.

Stomp Backend Client



- 다시 UbisamSubscriber.java 로 돌아와서 왼쪽 전구를 누른 후 Add unimplemented methods 를

  클릭하여 필요한 인터페이스를 구현해줍니다.

  아래 사진과 같이 구현됩니다.

```java
import com.fasterxml.jackson.databind.JsonNode;

import io.u2ware.common.stomp.client.WebsocketStompClient;
import io.u2ware.common.stomp.client.WebsocketStompClientHandler;

public class UbisamSubscriber implements WebsocketStompClientHandler{

    @Override
    public void handleFrame(WebsocketStompClient client, JsonNode message) {
        // TODO Auto-generated method stub
        throw new UnsupportedOperationException("Unimplemented method 'handleFrame'");
    }

}
```

- 그 다음 아래와 같이 코드를 작성해 줍니다.

```
import org.springframework.beans.factory.annotation.Autowir
ed;
import org.springframework.stereotype.Component;

import com.fasterxml.jackson.databind.JsonNode;

import io.u2ware.common.stomp.client.WebsocketStompClient;
import io.u2ware.common.stomp.client.WebsocketStompClientHa
ndler;
import io.u2ware.common.stomp.client.config.WebsocketStompP
roperties;

@Component
public class UbisamSubscriber implements WebsocketStompClie
ntHandler{

    @Autowired
    public WebsocketStompProperties properties;

    @Override
    public void handleFrame(WebsocketStompClient client, Js
onNode message) {
        System.out.println("message: " + message.toString
());
    }

    @Override
    public String getDestination() {
        return properties.getSubscriptions().get("ubisam");
    }
}
```

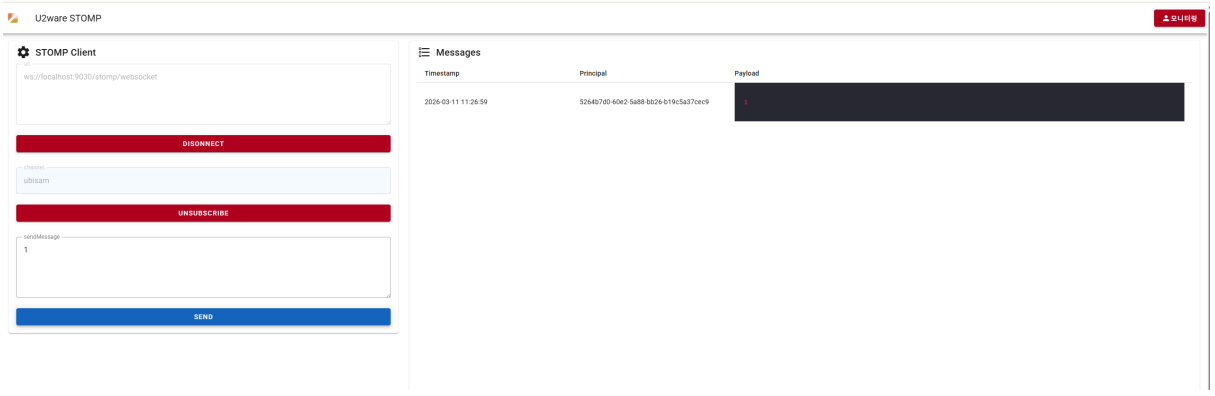
```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

import com.fasterxml.jackson.databind.JsonNode;

import io.u2ware.common.stomp.client.WebsocketStompClient;
import io.u2ware.common.stomp.client.WebsocketStompClientHandler;
import io.u2ware.common.stomp.client.config.WebsocketStompProperties;

@Component
public class UbisamSubscriber implements WebsocketStompClientHandler{

    @Autowired
    public WebsocketStompProperties properties;

    @Override
    public void handleFrame(WebsocketStompClient client, JsonNode message) {
        System.out.println("message: " + message.toString());
    }

    @Override
    public String getDestination() {
        return properties.getSubscriptions().get("ubisam");
    }
}
```

- 그 후 ./mvnw spring-boot:run 혹은 .\mvnw spring-boot:run 명령어를 통해 서버를 실행해 줍니다.

  아래와 같이 afterConnected: 가 뜨면 정상적으로 연결된 것입니다.

```
2026-03-11T11:23:25.740+09:00  INFO 13524 --- [example1] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2026-03-11T11:23:25.761+09:00  WARN 13524 --- [example1] [           main] org.hibernate.orm.deprecation            : HHH90000025: PostgreSQLDialect does not need to be specified explicitly using 'hib
 by default)
2026-03-11T11:23:25.779+09:00  INFO 13524 --- [example1] [           main] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
        Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
        Database driver: undefined/unknown
        Database version: 16.4
        Autocommit mode: undefined/unknown
        Isolation level: undefined/unknown
        Minimum pool size: undefined/unknown
        Maximum pool size: undefined/unknown
2026-03-11T11:23:26.456+09:00  INFO 13524 --- [example1] [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to
2026-03-11T11:23:26.509+09:00  INFO 13524 --- [example1] [           main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-03-11T11:23:26.888+09:00  WARN 13524 --- [example1] [           main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be
n-in-view to disable this warning
2026-03-11T11:23:27.376+09:00  INFO 13524 --- [example1] [           main] o.s.b.a.e.web.EndpointLinksResolver      : Exposing 1 endpoint beneath base path '/actuator'
2026-03-11T11:23:27.427+09:00  INFO 13524 --- [example1] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8080 (http) with context path '/'
2026-03-11T11:23:27.438+09:00  INFO 13524 --- [example1] [           main] com.ubisam.example1.Application          : Started Application in 4.044 seconds (process running for 4.317)
2026-03-11T11:23:32.567+09:00  INFO 13524 --- [example1] [ient-AsyncIO-16] cketStompClientConfiguration$Reconnector :    [/topic/ubisam]: com.ubisam.example1.stomp.ubisam.UbisamSubscriber@151f1734
2026-03-11T11:23:32.567+09:00  INFO 13524 --- [example1] [ient-AsyncIO-16] i.u.c.s.client.handlers.LoggingHandler   : [example1] afterConnected:
```

- 브라우저를 열어서 ubisam을 구독한 후 아무 메시지를 보내 테스트를 해봅니다.



- 메세지를 보낸 후 Backend를 터미널을 확인합니다.

  아래 사진과 같이 정상적으로 메세지를 받은 것을 확인하실 수 있습니다.

```
2026-03-11T11:26:48.430+09:00  INFO 12212 --- [example1] [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-03-11T11:26:48.490+09:00  INFO 12212 --- [example1] [           main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-03-11T11:26:48.978+09:00  WARN 12212 --- [example1] [           main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.ope
n-in-view to disable this warning
2026-03-11T11:26:49.453+09:00  INFO 12212 --- [example1] [           main] o.s.b.a.e.web.EndpointLinksResolver      : Exposing 1 endpoint beneath base path '/actuator'
2026-03-11T11:26:49.507+09:00  INFO 12212 --- [example1] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8080 (http) with context path '/'
2026-03-11T11:26:49.517+09:00  INFO 12212 --- [example1] [           main] com.ubisam.example1.Application          : Started Application in 4.01 seconds (process running for 4.221)
2026-03-11T11:26:54.603+09:00  INFO 12212 --- [example1] [lent-AsyncIO-16] cketStompClientConfiguration$Reconnector :   [/topic/ubisam]: com.ubisam.example1.stomp.ubisam.UbisamSubscriber@2446be5f
2026-03-11T11:26:54.603+09:00  INFO 12212 --- [example1] [lent-AsyncIO-16] ExceptionLoggingHandler  : [example1] afterConnected:
message: {"timestamp":1773196019483,"principal":"5264b7d0-60e2-5a88-bb26-b19c5a37cec9","payload":1}
```

- 다음으로는 Backend Server가 Client로 메시지를 전송해보겠습니다.

  아래와 같이 코드를 작성해줍니다.

  예시로 저희는 ubisam에서 메시지가 오면 robot으로 전송해보겠습니다.

```
import org.springframework.beans.factory.annotation.Autowir
ed;
import org.springframework.stereotype.Component;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ObjectNode;

import io.u2ware.common.stomp.client.WebsocketStompClient;
import io.u2ware.common.stomp.client.WebsocketStompClientHa
ndler;
import io.u2ware.common.stomp.client.config.WebsocketStompP
roperties;

@Component
public class UbisamSubscriber implements WebsocketStompClie
ntHandler{

    @Autowired
    public WebsocketStompProperties properties;

    @Autowired
    public ObjectMapper mapper;

    @Override
    public void handleFrame(WebsocketStompClient client, Js
```

```
onNode message) {
        System.out.println("message: " + message.toString
());

        ObjectNode data = mapper.createObjectNode();
        data.put("message", "hello");

        client.send("/app/robot", data);
    }

    @Override
    public String getDestination() {
        return properties.getSubscriptions().get("ubisam");
    }
}
```

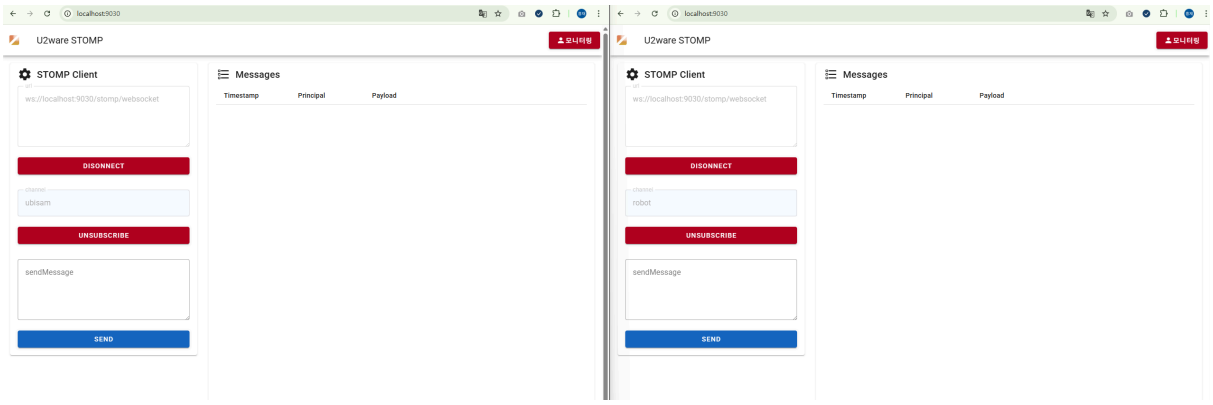
```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ObjectNode;

import io.u2ware.common.stomp.client.WebsocketStompClient;
import io.u2ware.common.stomp.client.WebsocketStompClientHandler;
import io.u2ware.common.stomp.client.config.WebsocketStompProperties;

@Component
public class UbisamSubscriber implements WebsocketStompClientHandler{

    @Autowired
    public WebsocketStompProperties properties;

    @Autowired
    public ObjectMapper mapper;

    @Override
    public void handleFrame(WebsocketStompClient client, JsonNode message) {
        System.out.println("message: " + message.toString());

        ObjectNode data = mapper.createObjectNode();
        data.put(propertyName: "message", v: "hello");

        client.send(destination: "/app/robot", data);
    }

    @Override
    public String getDestination() {
        return properties.getSubscriptions().get("ubisam");
    }
}
```

> 💡 /app/[채널] 은 보내고 싶은 채널을 적으면 됩니다.
> 만약 /app/ubisam 으로 보낼 경우 무한루프에 빠지게 되므로 주의가 필요합니다.

- 그 후 ./mvnw spring-boot:run 혹은 .\mvnw spring-boot:run 명령어를 통해 서버를 실행해 줍니다.

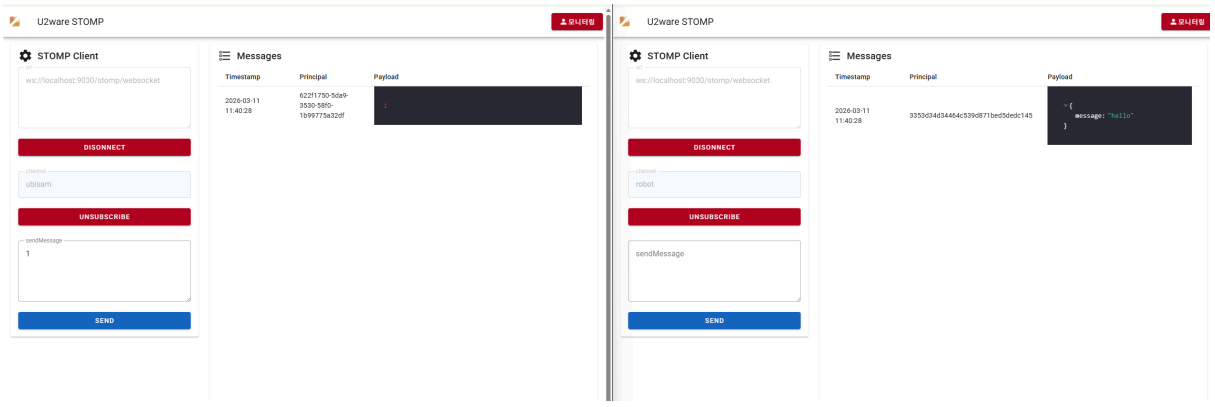
  테스트를 위해 아래 사진과 같이 2개의 브라우저를 연 후

  1개는 ubisam을 구독, 1개는 robot을 구독합니다.



- ubisam을 구독한 브라우저에서 메시지를 보내 테스트를 해봅니다.

  그럼 아래 사진과 같이 hello 가 도착하는걸 확인해볼 수 있습니다.



- 이번에는 메시지를 받았을 시 hello에 데이터를 추가해보겠습니다.

  example_hello 테이블에 name에는 payload 값을, email에는 principal 값을 넣어 보겠습니다.

  아래와 같이 코드를 작성해줍니다.

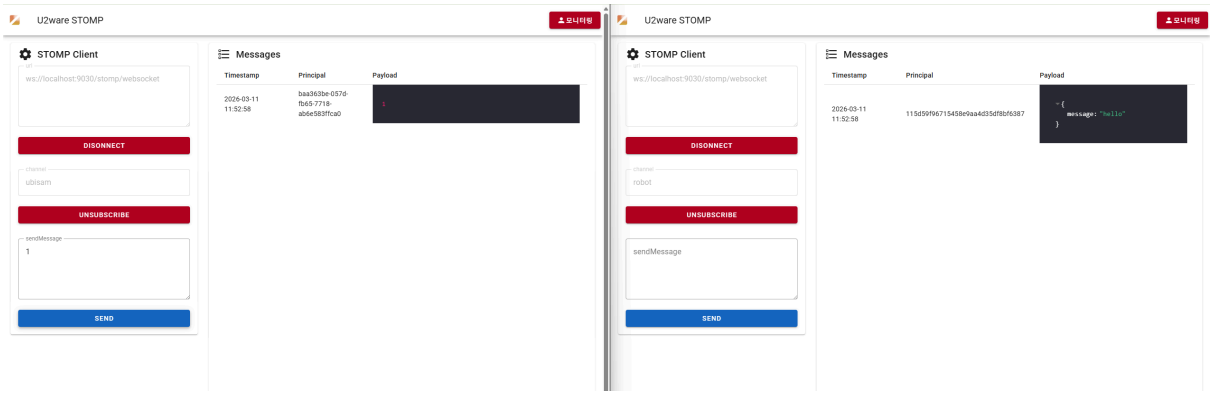
```
import org.springframework.beans.factory.annotation.Autowir
ed;
import org.springframework.stereotype.Component;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ObjectNode;
import com.ubisam.example1.api.helloes.HelloRepository;
import com.ubisam.example1.domain.Hello;

import io.u2ware.common.stomp.client.WebsocketStompClient;
import io.u2ware.common.stomp.client.WebsocketStompClientHa
ndler;
import io.u2ware.common.stomp.client.config.WebsocketStompP
roperties;

@Component
public class UbisamSubscriber implements WebsocketStompClie
ntHandler{

    @Autowired
    public WebsocketStompProperties properties;

    @Autowired
    public ObjectMapper mapper;

    @Autowired
    public HelloRepository helloRepository;

    @Override
    public void handleFrame(WebsocketStompClient client, Js
onNode message) {
        System.out.println("message: " + message.toString
());

        ObjectNode data = mapper.createObjectNode();
        data.put("message", "hello");
```

```
        Hello h = new Hello();
        h.setEmail(message.has("principal") ? message.get
("principal").asText() : null);
        h.setName(message.has("payload") ? message.get("pay
load").asText() : null);
        helloRepository.save(h);

        client.send("/app/robot", data);
    }

    @Override
    public String getDestination() {
        return properties.getSubscriptions().get("ubisam");
    }
}
```

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ObjectNode;
import com.ubisam.example1.api.helloes.HelloRepository;
import com.ubisam.example1.domain.Hello;

import io.u2ware.common.stomp.client.WebsocketStompClient;
import io.u2ware.common.stomp.client.WebsocketStompClientHandler;
import io.u2ware.common.stomp.client.config.WebsocketStompProperties;

@Component
public class UbisamSubscriber implements WebsocketStompClientHandler{

    @Autowired
    public WebsocketStompProperties properties;

    @Autowired
    public ObjectMapper mapper;

    @Autowired
    public HelloRepository helloRepository;

    @Override
    public void handleFrame(WebsocketStompClient client, JsonNode message) {
        System.out.println("message: " + message.toString());

        ObjectNode data = mapper.createObjectNode();
        data.put(propertyName: "message", v: "hello");

        Hello h = new Hello();
        h.setEmail(message.has(fieldName: "principal") ? message.get(fieldName: "principal").asText() : null);
        h.setName(message.has(fieldName: "payload") ? message.get(fieldName: "payload").asText() : null);
        helloRepository.save(h);

        client.send(destination: "/app/robot", data);
    }

    @Override
    public String getDestination() {
        return properties.getSubscriptions().get("ubisam");
    }
}
```

- 그 후 ./mvnw spring-boot:run 혹은 .\mvnw spring-boot:run 명령어를 통해 서버를 실행해 줍니다.

  아까와 동일하게 ubisam을 구독한 브라우저에서 메시지를 보내봅니다.



- 아래 사진과 같이 정상적으로 쿼리가 출력된 것을 확인하실 수 있습니다.

```
2026-03-11T11:48:32.003+09:00  INFO 26292 --- [example1] [ient-AsyncIO-14] i.u.c.s.client.handlers.LoggingHandler   : [example1] afterConnected:
message: {"timestamp":1773197578449,"principal":"baa363be-057d-fb65-7718-ab6e583ffca0","payload":1}
Hibernate:
    select
        nextval('example_hello_seq')
Hibernate:
    insert
    into
        example_hello
        (email, name, id)
    values
        (?, ?, ?)
2026-03-11T11:52:58.489+09:00 TRACE 26292 --- [example1] [ient-AsyncIO-14] org.hibernate.orm.jdbc.bind             : binding parameter (1:VARCHAR) <- [baa363be-057d-fb65-7718-ab6e583ffca0]
2026-03-11T11:52:58.490+09:00 TRACE 26292 --- [example1] [ient-AsyncIO-14] org.hibernate.orm.jdbc.bind             : binding parameter (2:VARCHAR) <- [1]
2026-03-11T11:52:58.490+09:00 TRACE 26292 --- [example1] [ient-AsyncIO-14] org.hibernate.orm.jdbc.bind             : binding parameter (3:BIGINT) <- [2152]
```

Stomp Frontend Client

- 먼저 npm 의존성을 추가합니다.

  아래 명령어를 터미널에 입력합니다.

> 💡 npm install stompjs

```
PROBLEMS 5   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POLYGLOT NOTEBOOK
PS C:\Users\user\Desktop\CHJ\notion\test\frontend1> npm install stompjs
```

- package.json으로 이동하여 정상적으로 설치됐는지 확인합니다.

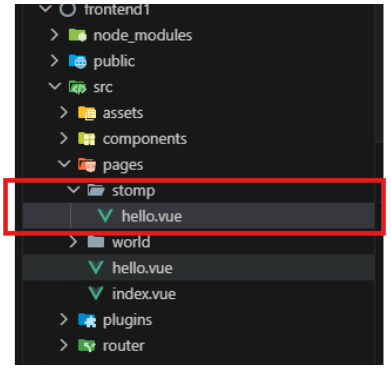
```
PS C:\Users\user\Desktop\CHJ\notion\test\frontend1> npm install stompjs
npm warn deprecated yaeti@0.0.6: Package no longer supported. Contact Su

added 19 packages, and audited 358 packages in 7s

103 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\user\Desktop\CHJ\notion\test\frontend1>
```

```
},
"dependencies": {
  "@fontsource/roboto": "^5.2.9",
  "@mdi/font": "7.4.47",
  "axios": "^1.13.6",
  "stompjs": "^2.3.3",
  "vue": "^3.5.26",
  "vue-i18n": "^11.2.7",
  "vue-router": "^4.6.4",
  "vuetify": "^4.0.0"
},
```

- 위와 오른쪽 사진과 같이

  추가됐으면 정상입니다.

- 이제 pages 폴더 밑에 stomp 폴더를 생성 후 hello.vue를 만듭니다.



- 그 후 아래와 같이 코드를 작성합니다.

```
<template>
    <h1>Hello World!</h1>
</template>
<script>
    import stompjs from 'stompjs';

    export default{
        name: 'Hello',
        mounted(){
            let url = 'ws://localhost:9030/stomp/websocke
t';
            let ws = stompjs.client(url);
            ws.connect({}, frame => {
                console.log('Connected :' + frame);
                ws.subscribe('/topic/robot', message => {
                    console.log('message: '+ message.body);
                });
            }, error => {
                console.error("error: "+ error);
            })
        },
    };
</script>
```

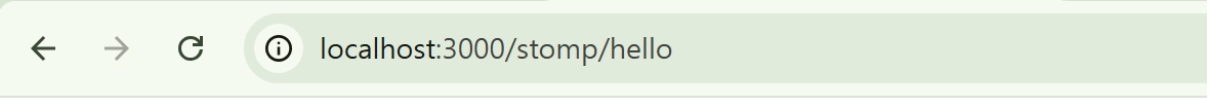
```vue
<template>
    <h1>Hello World!</h1>
</template>
<script>
    import stompjs from 'stompjs';

    export default{
        name: 'Hello',
        mounted(){
            let url = 'ws://localhost:9030/stomp/websocket';
            let ws = stompjs.client(url);
            ws.connect({}, frame => {
                console.log('Connected :' + frame);
                ws.subscribe('/topic/robot', message => {
                    console.log('message: '+ message.body);
                });
            }, error => {
                console.error("error: "+ error);
            })
        },
    };
</script>
```

- 이전 backend에서는 url 부문에 /websocket을 붙이지 않았지만

  이번 frontend는 붙여주셔야합니다.
- 그 후 npm run dev 혹은 npm run hello 로 서버를 열어줍니다.

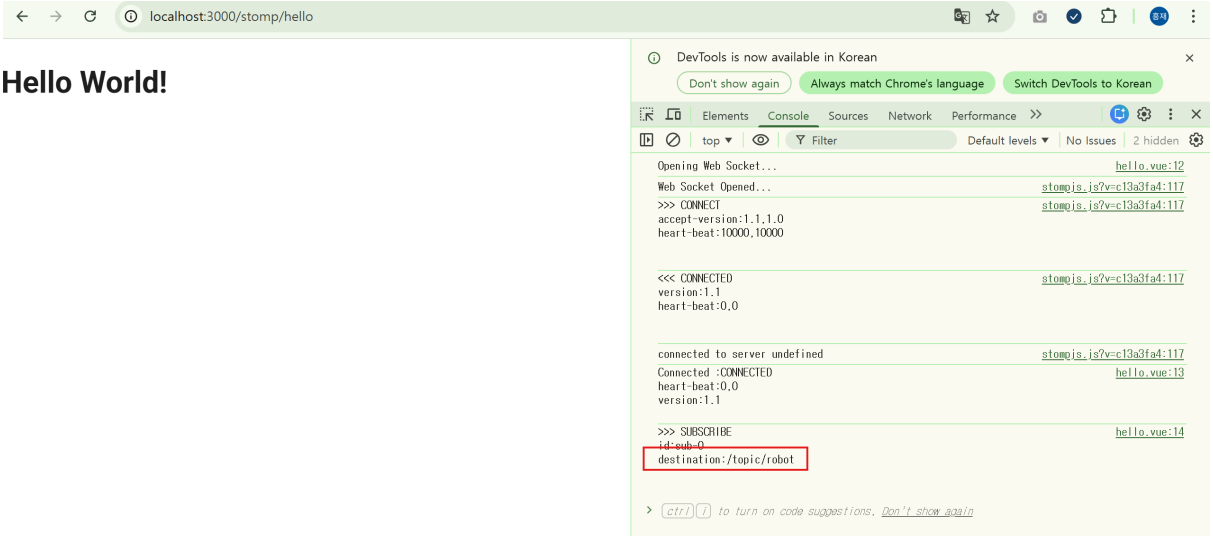
  COC 규약에 의거하여 http://localhost:3000/stomp/hello 로 이동해줍니다.

  아래 사진과 같이 Hello World! 가 출력되면 정상입니다.



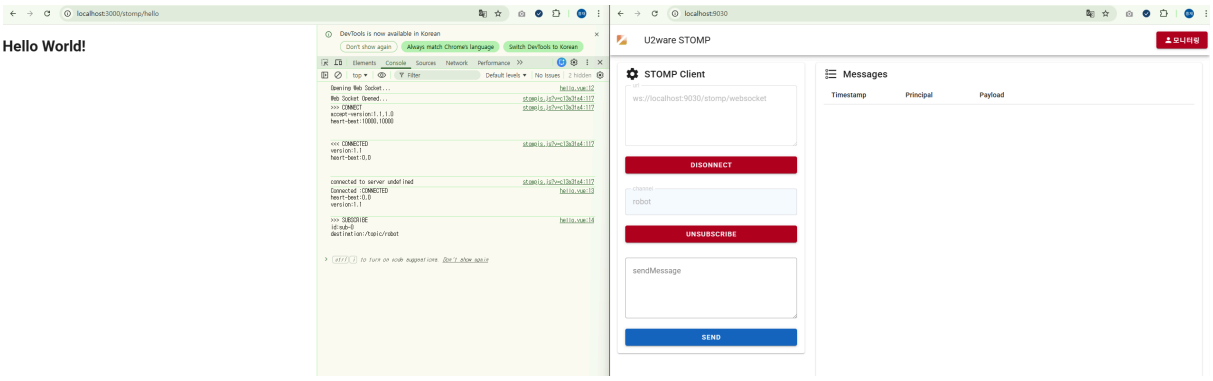
Hello World!

- 개발자도구 Console 을 확인하시면 robot을 구독했다는 메시지를 확인하실 수 있습니다.

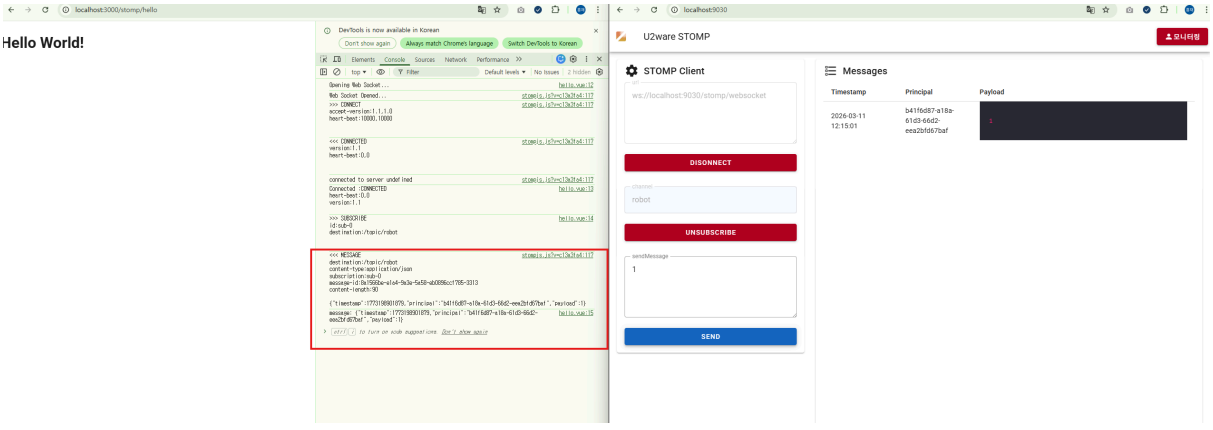


- 정상적으로 Frontend도 메시지가 오는지 테스트를 위해 robot을 구독하는 브라우저를 엽니다.

  아래 사진과 같이 robot을 구독해놓습니다.



- 이제 아무 메시지를 보내서 테스트를 해봅니다.



- 위 사진과 같이 오른쪽에서 1을 보내니 왼쪽에 console에 값이 출력되는 것을 확인하실 수 있습니다.

이렇게 3월 10일 세미나에 대한 내용 공유였습니다.

제 소스 파일은 github에 올려놨습니다.

https://github.com/ubisam-heung/myownstudy-backend

https://github.com/ubisam-heung/myownstudy-frontend

긴 글 이였는데도 읽어주셔서 감사합니다 😊